# Yenepoya University Commerce &Science Project – High Level Design on SmartCity ID — Secure Identity Framework


Student Name: Sayed Abdullah suadaan

Industry Mentor: Mr Shashank

Guided by: Ms Vijayalaxmi

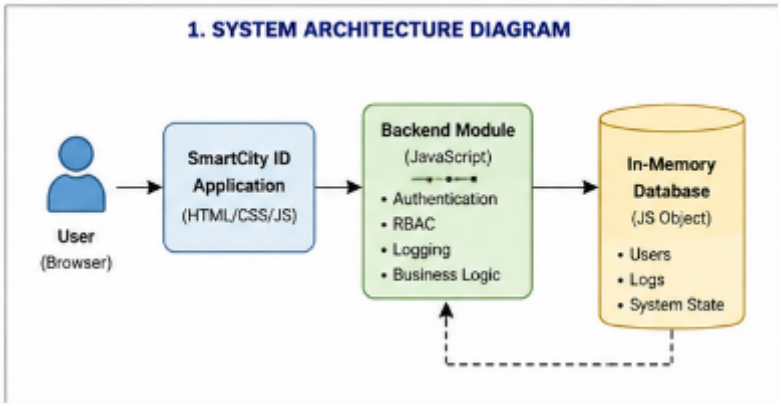
# Table of Contents


1. Introduction.....3
   - .1 Scope of the Document.....3
   - .2 Intended Audience.....
   3
   - .3 Definitions and Abbreviations.....3
2. System Overview.....4
   - 2.1 Project Background.....4
   - 2.2 Technology Stack.....4
   - 2.3 Objectives..... 4
3. System Design.....5
   - 3.1 Application
     Design..... 5 3.2
     Process Flow.....5
   - 3.3 InformationFlow.....
     Design.....
   - 3.5 Key Design Considerations.....7
   - 3.6 A
Catalogue.....7
   - Data Design.....8
   - 3.4 Data Model.....8
   - 3.5 Data Access Mechanism.....8
   - 3.6 Data Retention Policies.....9
   - 3.7 Data Migration.....9
4. Interfaces.....9
5. State and Session Management.....10
6. Caching.....10
7. Non-Functional Requirements.....10
   - 7.1 Security
     Aspects.....10
   - 7.2 Performance
     Aspects.....11
8. References.....1

# 1. Introduction

SmartCity ID is a browser-based Secure Identity Framework designed for a smart city ecosystem. It implements Role-Based Access Control (RBAC) to govern who can access what information and functionality based on their assigned role — either Admin or Citizen. The system is built entirely using HTML5, CSS3, and Vanilla JavaScript, functioning as a fully self-contained single-file application that requires no internet connection, no server, and no external dependencies.

The project demonstrates core principles of Identity and Access Management (IAM), including secure authentication, hashed credentials, role enforcement, activity logging, and a futuristic user interface that reflects smart city aesthetics. The system simulates real-world backend logic in JavaScript, equivalent to what would typically be implemented in Python or another server-side language.




## 1.1 Scope of the Document

This High Level Design (HLD) document describes the overall architecture, system components, data flow, user interfaces, and non-functional requirements for the SmartCity ID project. It covers:

- Authentication system — registration, login, logout with hashed password storage
- Role-based access control — Admin and Citizen roles with distinct dashboards
- Citizen dashboard — personal identity profile and read-only city status monitoring
- Admin control room — system overview, activity logs, and registered user management
- Backend simulation — Python-equivalent traffic logic, time conditions, and system state
- Security design — role enforcement, input validation, and session management
- UI design — futuristic dark theme optimised for desktop and mobile browsers

## 1.2 Intended Audience

This document is intended for the following stakeholders:

- Academic Evaluators and Faculty reviewing the project design
- Industry Mentors from IBM / NCS assessing technical design quality
- Developers who may extend or maintain the system in the future
- Quality Assurance Engineers designing test cases based on the system design

## 1.3 Definitions and Abbreviations

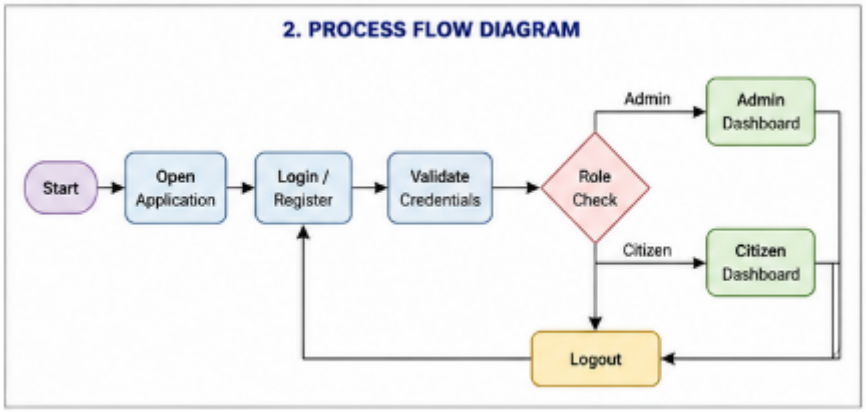
| Term | Full Form / Definition |
|---|---|
| HLD | High Level Design — overall system architecture document |
| RBAC | Role-Based Access Control — access governed by assigned roles |
| IAM | Identity and Access Management |
| Admin | System Administrator — has full system view and management access |
| Citizen | Registered end-user — has read-only access to personal info and city status |
| JS | JavaScript — programming language used for all logic |
| DOM | Document Object Model — browser-side HTML structure |
| SPA | Single Page Application — all pages managed in one HTML file |
| UI | User Interface |
| CSS | Cascading Style Sheets — styling and animations |
| HASH | Deterministic one-way password transformation function |
| DB | In-memory simulated database object within the BACKEND module |

# 2. System Overview

SmartCity ID addresses the challenge of controlled access in a digital smart city environment. In a smart city, multiple types of users interact with city systems — administrators who manage and monitor everything, and citizens who need access only to their personal records and public city status information. This project provides a secure, lightweight identity portal to simulate this access control scenario.

The system operates entirely on the client side within a single HTML file. There is no database server, no backend server, and no network calls — making it portable, offline-capable, and deployable on any device with a modern browser. Despite its simplicity, the design mirrors real-world IAM architecture through the separation of concerns between the BACKEND module

(business logic) and the APP module (controller/UI logic).




## 2.1 Project Background

The concept of a SmartCity requires secure management of citizen identities and administrative controls. As cities become increasingly digital, the need for reliable, role-based identity systems becomes critical. Citizens need to access their personal records securely, while administrators require a separate, privileged interface to monitor and manage city-wide systems. Without proper access control, sensitive city infrastructure data could be exposed to unauthorised users, leading to security vulnerabilities and loss of public trust.

Traditional identity management systems rely on server infrastructure, relational databases, network APIs, and authentication tokens. This project demonstrates the core principles of such a system — authentication, authorisation, auditing, and role separation — within a purely client-side implementation that can serve as an educational prototype or demonstration platform. The design mirrors real-world IAM architecture while remaining accessible without any server setup.

The project was developed as part of an academic programme in collaboration with IBM Innovation Centre for Education and NCS, focusing on applying industry-grade design thinking and documentation standards to a self-contained web application. The IBM HLD format was adopted to align the project documentation with professional industry standards used in enterprise software development.

A key design challenge was ensuring the application worked fully offline on mobile devices. Early versions relied on Google Fonts CDN and separate CSS/JS files, which caused layout failures when opened locally on mobile browsers. The final design embeds everything in a single HTML file with zero external dependencies, solving this real-world portability problem.

## 2.2 Technology Stack

| Layer | Technology | Purpose |
|---|---|---|
| Presentation | HTML5 | Page structure for all four views (Login, Register, Citizen, Admin) |
| Styling | CSS3 with CSS Variables | Futuristic dark theme, animations, responsive grid |

| | | |
|---|---|---|
| | | layout |
| Application Logic | Vanilla JavaScript (APP module) | Event handling, client-side routing, DOM updates, interval management |
| Business Logic | JavaScript (BACKEND module) | Authentication, hashing, role checks, traffic logic, log management |
| Data Storage | In-memory JS Object (DB) | Simulated user database — no persistent storage, cleared on refresh |
| Fonts | System Fonts (offline) | Courier New for display, Arial/system-sans for body — no CDN needed |

### 2.3 Objectives

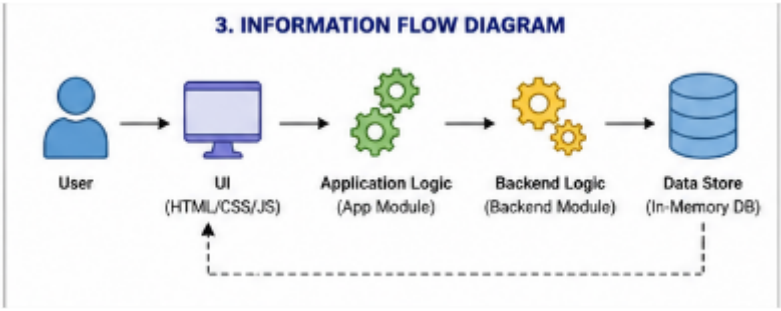
The SmartCity ID project was designed to meet the following primary and secondary objectives:

- Implement a secure login and registration system with hashed password storage — ensuring that plain-text passwords are never stored, displayed, or transmitted anywhere in the system.
- Enforce strict role-based access control — citizens and admins are routed to entirely separate dashboards after authentication, with admin-only functions guarded at both the UI layer and the business logic layer.
- Provide citizens with read-only access to their personal identity record — including full name, username, assigned role, email address, district, pincode, and account registration date — along with live city status monitoring.
- Provide administrators with a multi-tab control room — featuring a system overview panel, a real-time auto-refreshing audit log, and a complete registered users management table.
- Simulate real-world Python backend logic in JavaScript — including time-based traffic level computation (LOW/MEDIUM/HIGH based on hour of day), shift condition detection (MORNING/AFTERNOON/EVENING/NIGHT), and dynamic system state management.
- Deliver a fully offline, mobile-responsive, single-file application — that works by simply opening the HTML file in any browser on any device, with no installation, no server, and no internet connection required.
- Apply a futuristic, professional UI design — using dark backgrounds, neon cyan and green accent colors, animated grid overlays, glowing AI Brain orb, scan-line animations, and toast notifications that together create a convincing smart city aesthetics.
- Produce complete professional documentation — including this High Level Design document in the IBM format, covering all architecture, design decisions, data models, interfaces, security, and non-functional requirements.

## 3. System Design

The SmartCity ID system is designed as a Single Page Application (SPA) with four distinct views managed by a lightweight client-side router. The architecture separates the presentation layer (HTML/CSS), application controller (APP module), and business logic (BACKEND module). This separation ensures that business rules — such as role checks and password hashing — are

never bypassed through UI manipulation alone.




## 3.1 Application Design

The application consists of four pages, all residing within a single index.html file. Pages are shown and hidden by toggling the CSS class 'active'. The APP module manages routing, event listeners, and all DOM updates. The BACKEND module handles all business logic and exposes a safe public API.

| Page / View | ID | Accessible By | Purpose |
|---|---|---|---|
| Login Page | login-page | All users | Username and password authentication entry point |
| Register Page | register-page | New users | New citizen account creation with field validation |
| Citizen Dashboard | citizen-page | Citizens only | Personal identity profile and read-only city status |
| Admin Control Room | admin-page | Admin only | System overview, audit logs, and user management |

## 3.2 Process Flow

The following describes the end-to-end process flow for a user interacting with the SmartCity ID system:

- User opens the application — Login Page is displayed as the default view
  - If first-time user: clicks Register → completes registration form → redirected to Login
- If returning user: enters credentials on Login Page •
  Login validation is performed by BACKEND.login():
  - If username not found → error logged, 'User not found' alert shown
  - If password hash does not match → error logged, 'Incorrect password' alert shown
  - If credentials valid → user object returned (without password field)
- APP.handleLogin() reads the user role from the returned object: ◦ role === 'citizen' → initCitizenDashboard() called → Citizen Page shown
  - role === 'admin' → initAdminDashboard() called → Admin Page shown

- Citizen Dashboard runs auto-refresh intervals (status every 5s, clock every 1s)
- Admin Control Room runs auto-refresh intervals (status + logs every 4s)
- User clicks Logout → intervals cleared, currentUser set to null → Login Page shown

## 3.3 Information Flow

Information flows between the three layers of the system in a strictly controlled manner. The BACKEND module's DB object is private — it is never exposed directly to the APP module or the browser's global scope. All access is through the BACKEND public API.

| Direction | From | To | Data / Action |
|---|---|---|---|
| User → APP | Browser DOM (form inputs) | APP event handlers | Username, password, registration fields |
| APP → BACKEND | APP.handleLogin() | BACKEND.login() | Raw username and password string |
| BACKEND → APP | BACKEND.login() | APP.handleLogin() | { success, user } — user object with no password |
| APP → DOM | APP.initCitizenDashboard () | HTML elements | Personal info, city status values |
| BACKEND → APP | BACKEND.getSystemStat us() | APP update functions | Traffic, time, AI status, power, security level |
| BACKEND → APP | BACKEND.getLogs() | APP.renderLogs() | Array of log entry objects |
| BACKEND → APP | BACKEND.getUsers() | APP.renderUsers() | Array of user objects (no passwords) |

## 3.4 Components Design

The system is composed of the following key components:

| Component | Type | Description |
|---|---|---|
| BACKEND Module | JavaScript IIFE | Encapsulates entire DB and all business logic. Exposes: login, register, getSystemStatus, getLogs, getUsers, addLog |
| APP Module | JavaScript IIFE | Controls routing, event listeners, DOM updates, and interval management. Exposes: init() |
| **Component** | **Type** | **Description** |
| DB Object | Private JS Object | In-memory simulated database holding users array, logs array, system flags (emergencyActive, securityLevel, aiBrainStatus, systemPower) |

| | | |
|---|---|---|
| hashPassword() | Private function | Deterministic rolling hash — converts plain text password to HC$ prefixed hex string |
| getTrafficLevel() | Private function | Returns LOW/MEDIUM/HIGH based on current hour — simulates Python backend time logic |
| getTimeCondition() | Private function | Returns MORNING/AFTERNOON/EVENING/NIGHT_SHIFT based on current hour |
| showPage() | APP function | CSS-based router — hides all pages, shows target page with entrance animation |
| renderLogs() | APP function | Generates color-coded HTML log entries from BACKEND.getLogs() array |
| renderUsers() | APP function | Generates user management table rows from BACKEND.getUsers() array |

## 3.5 Key Design Considerations

The following design decisions were made to ensure the SmartCity ID system is secure, portable, maintainable, and accessible on all platforms including mobile devices:

- Single File Architecture: The entire application — HTML, CSS, and JavaScript — is contained in one self-contained file. This eliminates all dependency on relative file paths or local web servers, which commonly cause loading failures when opening HTML projects on mobile devices or sharing via WhatsApp, email, or USB transfer.
  - Benefit: Works instantly on any device — double-click to open, no installation or internet needed.
  - Benefit: Eliminates 'file not found' errors caused by broken CSS or JS file paths on different operating systems.
- IIFE Pattern for Encapsulation: Both BACKEND and APP modules use the Immediately Invoked Function Expression (IIFE) pattern. The entire DB object and all internal helper functions are scoped inside the IIFE closure, making them inaccessible from the browser console or external scripts.
  - Benefit: Simulates server-side data privacy — citizens cannot read or modify the DB object through browser developer tools.
  - Benefit: Prevents accidental variable name collisions between the two modules.
- No Persistent Storage by Design: localStorage and sessionStorage are intentionally avoided. All session state is held in module-level closure variables that are automatically cleared when the tab is closed or refreshed.
  - Security reason: Avoids XSS-based session theft — an injected script cannot read session tokens from storage APIs.
  - Privacy reason: No personally identifiable data is left on the device after the session ends.
- Role Enforcement at Multiple Layers: User role is verified at two independent checkpoints

— (1) the APP-level router immediately after login, and (2) inside each admin-only BACKEND function before execution.
  - This dual-layer enforcement means that even if a malicious user somehow bypassed the UI router, the BACKEND function would still reject the call with an ACCESS DENIED response.
- Offline First Design: All fonts, stylesheets, scripts, and assets are embedded inline — zero external network calls are made. The application runs identically with or without an internet connection.
  - This was a deliberate response to the real user issue encountered where Google Fonts CDN calls failed on mobile devices without internet, breaking the entire layout.
- Mobile-Responsive Layout: CSS Grid and Flexbox are used throughout with media query breakpoints at 600px and 380px. Input fields are set to font-size: 16px to prevent iOS Safari's automatic zoom behaviour on tap.
  - Touch targets for all buttons and interactive elements are set to a minimum height of 44px following Apple's Human Interface Guidelines.
- Audio Feedback via Web Audio API: Login success and failure are accompanied by distinct audio beeps generated programmatically through the Web Audio API — no audio files are required. The call is wrapped in a try-catch to silently handle browsers that block auto-play audio.

### 3.6 API Catalogue

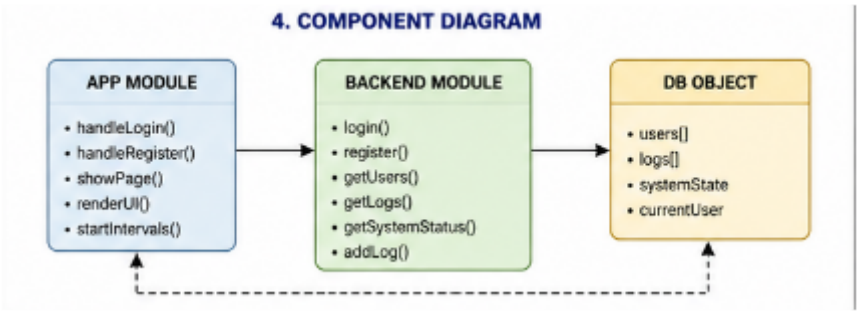
The BACKEND module exposes a clean public API. All internal helper functions (hashPassword, generateSensors, getTrafficLevel, getTimeCondition) are private and inaccessible outside the IIFE. The following table documents all public functions:

| Function | Parameters | Returns | Access |
|---|---|---|---|
| login(username, password) | username: string, password: string (plain) | { success: bool, user?: object, message?: string } | Public |
| register(username, password, fullName, email, district, pincode) | All registration fields | { success: bool, message: string } | Public |
| getSystemStatus() | None | Status object: traffic, time, power, security, AI, timestamp | Public |
| getLogs() | None | Array of log entry objects (max 50) | Public |
| getUsers() | None | Array of user objects — password field stripped | Public |
| addLog(type, message, level) | type: string, message: string, level: INFO\|WARNING\|CRITICAL | void — prepends to DB.logs | Public |

## 4. Data Design

All data in the SmartCity ID system is stored in-memory within the BACKEND module's private

DB object. No external database, file system, or browser storage is used. Data is structured into three logical collections: the users array, the logs array, and system state flags.




## 4.1 Data Model

The following tables describe the data structures used within the system.

### User Object Schema:

| Field | Type | Required | Constraints | Example |
|---|---|---|---|---|
| id | Integer | Yes | Auto-incremented from 1 | 1 |
| username | String | Yes | Min 3 chars, unique in DB.users | john_doe |
| password | String | Yes | Stored as HC$ hash, never plain | HC$00A1B2C3D4E5 |
| role | String | Yes | Enum: 'admin' or 'citizen' | citizen |
| fullName | String | Yes | Non-empty string | John Doe |
| email | String | No | Auto-generated if not provided | john.doe@smartcity.gov |
| district | String | No | Kerala or Karnataka district | Kasaragod, Kerala |
| pincode | String | No | 6 digits if provided | 671121 |
| joined | String | Yes | ISO format YYYY-MM-DD | 2025-04-15 |

### Log Entry Schema:

| Field | Type | Description |
|---|---|---|
| id | Integer | Auto-incremented log entry number |
| timestamp | String | Time of event from toLocaleTimeString() |
| date | String | Date of event from toLocaleDateString() |

| | | |
|---|---|---|
| type | String | Category: SYSTEM, LOGIN, LOGIN_FAIL, REGISTER, LOGOUT, EMERGENCY, SECURITY, AI_BRAIN |
| message | String | Human-readable description of the event |
| level | String | Severity level: INFO, WARNING, or CRITICAL |

## 4.2 Data Access Mechanism

All data access follows the principle of least privilege. The internal DB object is completely private to the BACKEND IIFE and cannot be accessed from outside. The following access rules apply:

- Read access to user data: only through BACKEND.getUsers(), which strips the password field from every user object before returning the array.
- Read access to logs: only through BACKEND.getLogs(), which returns a copy of the log array.
- Write access to users: only through BACKEND.register() and BACKEND.login(), which hash passwords before storage and comparison respectively.
- Write access to logs: only through BACKEND.addLog(), called internally by all operations that require auditing.
- System state changes: only through admin-verified functions. Each function checks adminUser.role === 'admin' before executing and returns an ACCESS DENIED error if the check fails.

## 4.3 Data Retention Policies

Since the application uses in-memory storage exclusively, all data is ephemeral — it exists only during the current browser session. When the browser tab is closed or the page is refreshed, all user registrations, logs, and system state are reset to the initial state defined in the DB object. This is a deliberate design decision that prioritises privacy and security over persistence.

- Log retention: Maximum 50 entries. When the log count exceeds 50, the oldest entry is automatically removed (DB.logs.pop()) to maintain the cap. This prevents unbounded memory growth during long sessions and ensures the log container always displays the most recent and relevant activity.
- User retention: Users registered during a session are stored in DB.users for the duration of that session only. They are not persisted to any storage medium. On page reload, the system returns to the two default accounts: admin and john_doe.
- System state: Flags like emergencyActive (default: false), securityLevel (default: ALPHA), and aiBrainStatus (default: ONLINE) all reset to their default values on every page load, ensuring the system always starts in a known safe state.
- Password data: The hashPassword() function is deterministic — the same input always produces the same hash. This means login will still work correctly after a page refresh, as the stored hashed password and the freshly hashed login input will match.

If this application were deployed as a production system, data retention would be governed by the applicable data protection regulations, including the Information Technology Act 2000 (India), GDPR (if serving EU citizens), and the organisation's own data lifecycle management policies. User data would require encrypted storage, automated purging schedules, and

documented retention periods.

### 4.4 Data Migration

This system does not involve data migration in the traditional sense, as there is no persistent storage or external database. All data is initialised fresh on every page load from the hard-coded default values in the DB object. However, the system is designed with migration-readiness in mind — the BACKEND module's clean API surface means that swapping the in-memory DB for real database calls would require changes only within the BACKEND module, with no changes needed in the APP module or HTML. If this system were extended to a full production implementation, the following migration steps would be required:

- User data migration: The DB.users array would be replaced with API calls to a secure backend (Node.js/Express, Django, or Spring Boot) connected to a relational database such as PostgreSQL or MySQL. All existing demo passwords would need to be re-hashed using bcrypt (cost factor 12) or Argon2id before being stored.
- Password upgrade: The current deterministic rolling hash (HC$ prefix) is suitable only for demonstration. Production migration would require a one-time password reset campaign or a hash upgrade path where users are re-hashed on their next successful login.
- Log data migration: The DB.logs array would be replaced with calls to a structured logging service. Recommended options include Elasticsearch + Kibana for log analysis, AWS CloudWatch for cloud deployments, or a SIEM (Security Information and Event Management) system for compliance-driven environments.
- System state migration: In-memory state flags (emergencyActive, securityLevel, aiBrainStatus) would be migrated to a distributed state store such as Redis, allowing multiple admin sessions to share and observe the same system state in real time.
- District and pincode data: The static dropdown of Kerala and Karnataka districts embedded in the HTML would be migrated to a reference data API or a maintained lookup table in the database, allowing new districts to be added without modifying the application code.

## 5. Interfaces

The SmartCity ID system exposes four distinct user interfaces, each tailored to its intended audience and access level. All interfaces share the same futuristic dark theme with consistent typography, color palette, and animation language. The interface is built entirely with HTML5 and CSS3 — no UI framework such as React, Angular, or Bootstrap is used — keeping the application lightweight and dependency-free.

Navigation between pages is handled exclusively by the APP module's showPage() function, which uses CSS class toggling rather than URL routing. This means users cannot navigate directly to a protected page by typing a URL — they must go through the authentication flow every time. The active page has the CSS class 'active' applied, while all other pages have display:none, preventing any content from being visible or accessible outside of the intended flow.

| Interface | Accessible By | Key Elements |
|---|---|---|

| | | |
|---|---|---|
| Login Page | All users | Username field, password field with toggle, Authenticate button, alert box for success/error, link to Register page, demo credentials display |
| Register Page | New users | Full name, username, district dropdown (14 Kerala + 30 Karnataka), pincode, email, password, confirm password fields with validation alerts |
| Citizen Dashboard | Citizens | Welcome banner, personal identity table (name/role/email/district/pincode/joined), city status stats (AI Brain, Power, Security, Traffic), emergency banner, live clock, logout button |
| Admin Control Room | Admins only | Status strip (AI, Power, Security, Traffic), three tabs — Overview (emergency state, user count, AI orb), Logs (color-coded audit log, auto-refresh 4s), Users (full registered users table) |

The interface communicates system state visually through:

- Color coding: Traffic levels use green (LOW), amber (MEDIUM), and red (HIGH) coloring.
- Animated AI Brain orb: breathing glow animation indicates ONLINE status.
- Emergency banner: pulsing red banner appears on citizen dashboard when admin activates emergency override.
- Toast notifications: slide-in notifications appear on admin actions with color-coded severity.
- Shake animation: login form shakes horizontally on incorrect credentials.
- Scan line animation: continuous scan line sweeps the page to reinforce security context.

## 5.1 Login Page Interface

The Login Page is the default entry point for all users. It presents a centered authentication card styled with corner bracket decorators, a top-edge animated scan-line, and a secure badge reading '256-BIT ENCRYPTED CONNECTION — SECURE SESSION ACTIVE'. The form contains two input fields with a password visibility toggle (eye icon button), and an Authenticate button that triggers both visual and audio feedback.

Incorrect credentials produce a red shake animation on the form container and a red error alert for 4 seconds.

Successful login shows a green alert and triggers a page transition to the role-appropriate dashboard after a 1-second delay. A demo credentials block at the bottom shows the admin and citizen test logins for evaluators.

## 5.2 Registration Page Interface

The Registration Page allows new citizens to self-enroll. The district field is a grouped HTML select dropdown organised by state — 14 districts under the Kerala group and 30 districts under Karnataka. Each district value is stored in the format 'District Name, State' for display clarity in the user profile table. The pincode field validates against the regex /^[0-9]{6}$/ before the form reaches the BACKEND. All password fields have eye-toggle buttons. On successful registration, a success alert appears and the user is automatically redirected to the Login Page after 1.5 seconds.

## 5.3 Citizen Dashboard Interface

The Citizen Dashboard provides a read-only view of two categories of data. First, the Personal Identity Record section renders all profile fields — username, role, full name, email, district, pincode, and join date — in a formatted two-column table. Second, the City Status section displays four auto-refreshing metric cards showing AI Brain Status (with pulsing green status dot), System Power percentage, Security Level, and Traffic Level. Traffic Level text is color-coded using CSS classes: green for LOW, amber for MEDIUM, and red for HIGH. All four metrics are polled from BACKEND.getSystemStatus() every 5 seconds via setInterval. If an admin triggers the emergency override, a pulsing red emergency banner automatically appears at the top of the citizen dashboard on the next poll cycle without requiring any page reload.

## 5.4 Admin Control Room Interface

The Admin Control Room is the most feature-rich view, accessible only to users with role === 'admin'. At the top sits a 4-card status strip showing AI Brain, System Power, Security Level, and Traffic Level — identical to the citizen view but used here as a quick-glance header. Below this, three tabs organise the main content.

The Overview tab shows a 3-card grid: Emergency State (STANDBY in green or ACTIVE in pulsing red), an animated AI Brain orb with breathing glow effect, and the total registered user count. The Logs tab renders a scrollable list of up to 50 color-coded audit log entries — blue for INFO, amber for WARNING, red for CRITICAL — which auto-refresh every 4 seconds. The Users tab shows a complete registered users table with columns: ID, Username, Full Name, Role (with colored badge), District, Pincode, and Joined date.

| Visual Element | Trigger Condition | Effect |
|---|---|---|
| Traffic color coding | getTrafficLevel() result | Text: green (LOW) / amber (MEDIUM) / red (HIGH) with glow |
| AI Brain orb pulse | aiBrainStatus is ONLINE | 3-second breathing glow animation via CSS keyframes |
| Emergency banner | DB.emergencyActive === true | Pulsing red banner on citizen page,<br>1.5s animation cycle |
| Toast notification | Admin control action fires | Slide-in from bottom-right, colorcoded, auto-dismiss 3s |
| Form shake | Wrong login / validation failure | Horizontal shake animation, 0.5s CSS keyframe |

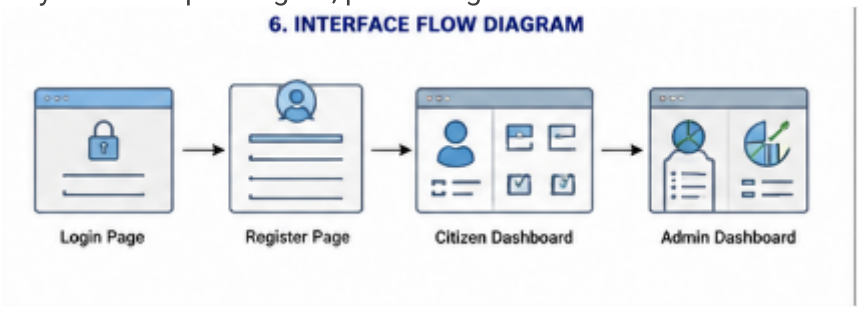
| | | |
|---|---|---|
| Scan-line overlay | Always on auth pages | Glowing line sweeps full page height every 6 seconds |
| Status dot pulse | Metric cards and badges | 7px circle fades between opacity 1 and 0.5 every 2s |
| Page fade-slide | After login or logout completes | opacity 0→1 and translateY 20px→0 over 0.5 seconds |
| Input focus glow | User clicks into any input field | 3px ring glow using box-shadow rgba(0,207,255,0.1) |
| Card hover lift | Mouse hover over dashboard cards | translateY(-1px) + stronger border glow on hover |

## 6. State and Session Management

Session state is managed entirely within the APP module's closure variable currentUser. This variable holds the authenticated user object (without the password field) for the duration of the session. There is no use of cookies, localStorage, sessionStorage, or any browser persistence mechanism — this is a deliberate security and portability decision that ensures zero data leakage between sessions.

The SmartCity ID system operates as a stateless client-side application from the perspective of the browser's persistence APIs. Every visit to the application starts a fresh session. This design is intentional — it mirrors the behaviour of time-limited session tokens used in production systems, where sessions expire and users must re-authenticate.

The application manages three types of real-time state: (1) user authentication state, (2) system status polling state, and (3) UI timer state. All three are held in the APP module closure and are fully cleaned up on logout, preventing resource leaks and stale data display.




The following state variables are maintained in the APP module closure:

| Variable | Type | Initial Value | Description |
|---|---|---|---|
| currentUser | Object \| null | null | Holds authenticated user. Set on login, cleared on logout or page refresh. |
| sensorInterval | Timer \| null | null | setInterval ID for status/sensor refresh. Cleared on logout. |
| clockInterval | Timer \| null | null | setInterval ID for live clock (1 second). Cleared on logout. |

| Variable | Type | Initial Value | Description |
|---|---|---|---|
| statusInterval | Timer \| null | null | setInterval ID for admin status+log refresh (4 seconds). Cleared on logout. |

On logout, the APP.handleLogout() function clears all three intervals using clearInterval() and sets currentUser to null before redirecting to the login page. This ensures no stale data or background processes continue after logout. A logout event is also written to the audit log via BACKEND.addLog() before the session is cleared, ensuring the logout action is captured even though the session ends immediately after.

A critical aspect of this design is that the interval IDs (sensorInterval, clockInterval, statusInterval) are stored in the APP closure and shared across functions. Without clearing these intervals on logout, they would continue running in the background, potentially updating DOM elements from the previous session or causing JavaScript errors when those elements no longer exist in the active page.

The system does not implement session timeout by design — the session remains active as long as the browser tab is open. In a production system, a 15–30 minute inactivity timeout would be recommended, implemented using a resetable setTimeout that triggers handleLogout() after the inactivity period expires.

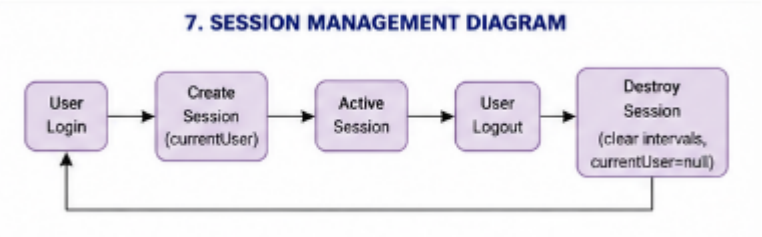
## 7. Caching

The SmartCity ID system does not implement application-level caching, as all data is already held inmemory within the BACKEND module. Since the DB object is a JavaScript object in RAM, all read operations — getLogs(), getUsers(), and getSystemStatus() — are effectively O(1) or O(n) in-memory reads with no I/O overhead, disk access, or network latency. The need for a caching layer simply does not arise in this architecture.

The system status values (traffic level, time condition, timestamp) are recomputed fresh on every call to BACKEND.getSystemStatus() using the current system clock (new Date()). This means the displayed city status is always accurate to the current moment, with no stale cache to invalidate or expire. The auto-refresh intervals (4 seconds for admin, 5 seconds for citizens) are tuned to balance update frequency against CPU load.

The application also benefits from the browser's built-in HTTP cache when served from a web server — since all assets are embedded in a single HTML file, the entire application is cached after the first load. Subsequent visits load instantly from the browser cache with zero network requests, making the application effectively a cached offline-ready web app.

If this system were extended to a production environment with a real backend and external API calls, the following caching strategies would be recommended:




| Data | Cache Store | TTL | Invalidation Strategy |
|---|---|---|---|

| | | | |
|---|---|---|---|
| Session tokens | Redis | 30 min sliding | Token deleted on explicit logout |
| System status | App memory | 5–10 seconds | Auto-expire; recomputed |
| **Data** | **Cache Store** | **TTL** | **Invalidation Strategy** |
| | | | on next read |
| User profiles | Redis / Memcached | 10 minutes | Invalidated on profile update |
| Audit logs | App memory ring | Until > 50 items | Oldest entry dropped on overflow |
| HTML/CSS/JS | CDN edge cache | 1 year | Cache-bust via deployment versioning |

# 8. Non-Functional Requirements

Non-functional requirements (NFRs) define the quality attributes and operational constraints of the SmartCity ID system. These requirements are as important as functional features — they determine how reliably, securely, and efficiently the system operates under real-world conditions. The following NFRs have been identified and addressed in the design.

## 8.1 Security Aspects

Security is the primary non-functional concern for an identity and access management system. The SmartCity ID design incorporates the following security controls:

| Security Control | Implementation | Threat Mitigated |
|---|---|---|
| Password Hashing | Deterministic rolling hash (HC$ prefix hex string). Plain text is never stored. | Credential exposure if DB is read by unauthorized party |
| Password Comparison | Input is hashed before comparison — plain text password is never compared directly. | Timing attacks and plain-text comparison vulnerabilities |
| Role Enforcement (UI) | APP router checks role after login and routes to correct page. Citizens cannot navigate to admin -page. | Unauthorized admin page access via URL manipulation |
| Role Enforcement (API) | Every admin-only BACKEND function checks adminUser.role === 'admin' before executing. | Privilege escalation via direct JS console function calls |
| Safe Data Return | getUsers() and login() strip the password field before returning user objects. | Accidental password exposure through API responses |

| | | |
|---|---|---|
| No External Requests | Zero CDN, API, or font calls — all assets embedded inline. | Man-in-the-middle attacks, CDN supply chain injection |
| Session in Closure | currentUser held in IIFE closure — not in localStorage or global scope. | XSS-based session token theft from browser storage |
| Audit Trail | addLog() called on every auth event with timestamp, type, and severity. | Lack of accountability and forensic evidence trail |
| Input Validation | All form fields validated in APP before BACKEND call; BACKEND re-validates independently. | SQL injection equivalents, malformed data attacks |
| No Auto-persist | Session cleared on tab close / refresh | Session hijacking from shared or |
| **Security Control** | **Implementation** | **Threat Mitigated** |
| | — no remember-me functionality. | public devices |

## 8.2 Performance Aspects

Performance requirements ensure the SmartCity ID system responds quickly and runs smoothly across all supported devices, including low-powered mobile devices. Since the application is entirely client-side with in-memory data, performance is inherently strong — the main considerations are animation smoothness, render efficiency, and file size.

| Requirement | Target | How Achieved |
|---|---|---|
| Login response time | < 100ms | In-memory hash comparison and lookup — zero network round-trip or disk access |
| Registration response | < 100ms | Array push with hash computation — executes entirely in RAM |
| Dashboard initial load | < 200ms | DOM innerHTML assignments only — no framework reconciliation or virtual DOM diff |
| Status auto-refresh | Every 4–5s | setInterval reads in-memory state object — negligible CPU cost per cycle |
| Log render (50 entries) | < 50ms | Single templateString.join map into innerHTML — one DOM write operation total |
| Users table render | < 30ms | Array.map to innerHTML string — O(n) single pass, typically <10 users |

| | | |
|---|---|---|
| Application file size | < 80 KB | Single self-contained HTML file with no external libraries or frameworks |
| CSS animation frame rate | 60 fps target | All animations use CSS transform and opacity — GPU-composited, no layout reflow |
| Mobile responsiveness | < 300ms touch | Touch-action: manipulation removes 300ms tap delay on mobile browsers |
| Offline load time | Instant | Zero external dependencies — entire app loads from local filesystem with no HTTP |
| Memory footprint | < 5 MB RAM | No framework overhead — plain JS objects and DOM nodes only |
| Interval cleanup | On every logout | clearInterval() called for all 3 intervals — no background CPU drain after logout |

## 8.3 Usability Requirements

The system is designed to be intuitive for both technical administrators and non-technical citizen users. The following usability standards are met:

| Requirement | Implementation |
|---|---|
| Error clarity | All form errors display plain-English messages (not error codes) in a color-coded alert visible immediately below the form |
| Visual feedback | Every user action — login, logout, registration, button click — produces immediate visual feedback within 100ms |
| Audio feedback | Distinct tones for login success (880Hz), failure (220Hz), and logout (440Hz) via Web Audio API — no audio files needed |
| Password security UX | Password fields have a toggle button to show/hide characters — prevents typos without exposing credentials unnecessarily |
| Mobile touch targets | All buttons and interactive elements have a minimum height of 44px as per Apple Human Interface Guidelines |
| iOS zoom prevention | All form inputs set to font-size: 16px — prevents iOS Safari automatic viewport zoom on input tap |

| | |
|---|---|
| Responsive layout | CSS Grid with breakpoints at 600px and 380px — dashboard grids collapse from 4-column to 2-column on mobile |
| Tab navigation | Admin tabs scroll horizontally on small screens with hidden scrollbar and touch-scroll support |
| Session transparency | Live clock displayed in all dashboards so users know the session is active and can see the current time |
| Logout visibility | Logout button prominently placed in the top-right of every dashboard header — always accessible |

## 8.4 Compatibility Requirements

| Platform | Browser | Notes |
|---|---|---|
| Desktop Windows | Chrome 90+, Edge 90+, Firefox 88+ | Full feature support — all CSS animations and Web Audio API available |
| Desktop macOS | Safari 14+, Chrome 90+ | Full feature support — tested on Safari which has stricter AudioContext policies |
| Mobile Android | Chrome for Android 90+ | Responsive layout confirmed — tap targets and horizontal tab scroll verified |
| Mobile iOS | Safari 14+ on iPhone and iPad | font-size:16px prevents zoom; AudioContext requires user gesture first tap |
| Offline use | All browsers — file:// protocol | No internet required — open directly as local HTML file with no server |
| Screen sizes | 320px to 2560px width | Tested at 320px (small phone), |
| Platform | Browser | Notes |
| | | 768px (tablet), 1440px (desktop) |

# 9. References

| # | Reference | Source / Link |
|---|---|---|
| 1 | IBM Innovation Centre for Education — HLD Format Template | HLD_FORMAT_-_IBM.docx (provided) |
| 2 | MDN Web Docs — Web Audio API | developer.mozilla.org/en-US/docs/Web/API/Web_Audio_API |

| | | |
|---|---|---|
| 3 | MDN Web Docs — Document Object Model | developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model |
| 4 | MDN Web Docs — IIFE (Immediately Invoked Function Expression) | developer.mozilla.org/en-US/docs/ Glossary/IIFE |
| 5 | OWASP Authentication Cheat Sheet | cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html |
| 6 | NIST SP 800-63B — Digital Identity Guidelines | pages.nist.gov/800-63-3/sp80063b.ht ml |
| 7 | CSS-Tricks — A Complete Guide to Custom Properties | css-tricks.com/a-complete-guide-tocu stom-properties/ |
| 8 | W3C — HTML5 Specification | html.spec.whatwg.org |
| 9 | Role-Based Access Control (RBAC) — NIST Definition | csrc.nist.gov/projects/role-basedacce ss-control |
| 10 | Kerala Districts — Government of Kerala Official Portal | kerala.gov.in |
| 11 | Karnataka Districts — Government of Karnataka Official Portal | karnataka.gov.in |


*— End of High Level Design Document —*